# Étude de la fluidification d’un automate cellulaire représentant un feu de forêt



**Martin CHAUBET**

# Quelques informations sur les feux de forêt

- 9/10 des feux sont d’origine humaine.

- 100 000 kW/m pour les plus gros feux. Un front de feu d’une puissance de 10 000 kW/m et de longueur 1 km dissipe en deux heures une énergie de 721 012 J = égale à celle de la bombe atomique d’Hiroshima.

- Pour les risques élevés, il faut compter environ 10 min après la détection.
- Moins de 1 hectare brûlé avant la première intervention.

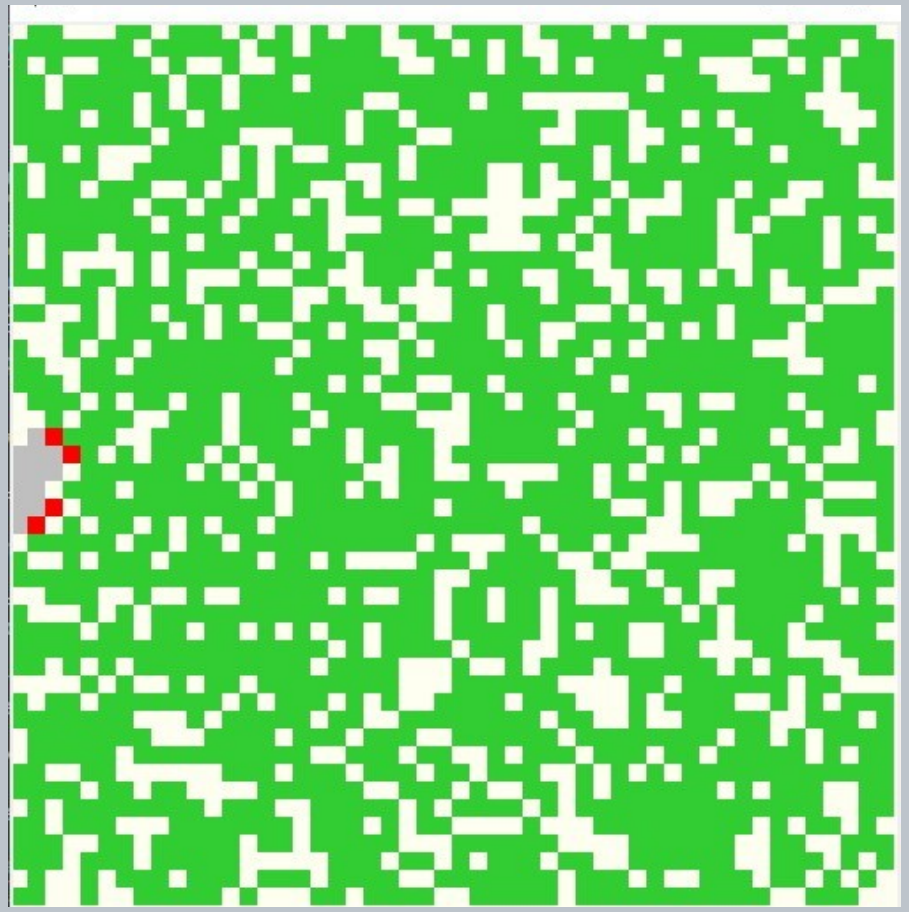
- Depuis 1995, les plans de prévention des risques d'incendies de forêt (PPRIF), tentent de prédire et d’empêcher les feux de forêts en fonction de la structures des sols.

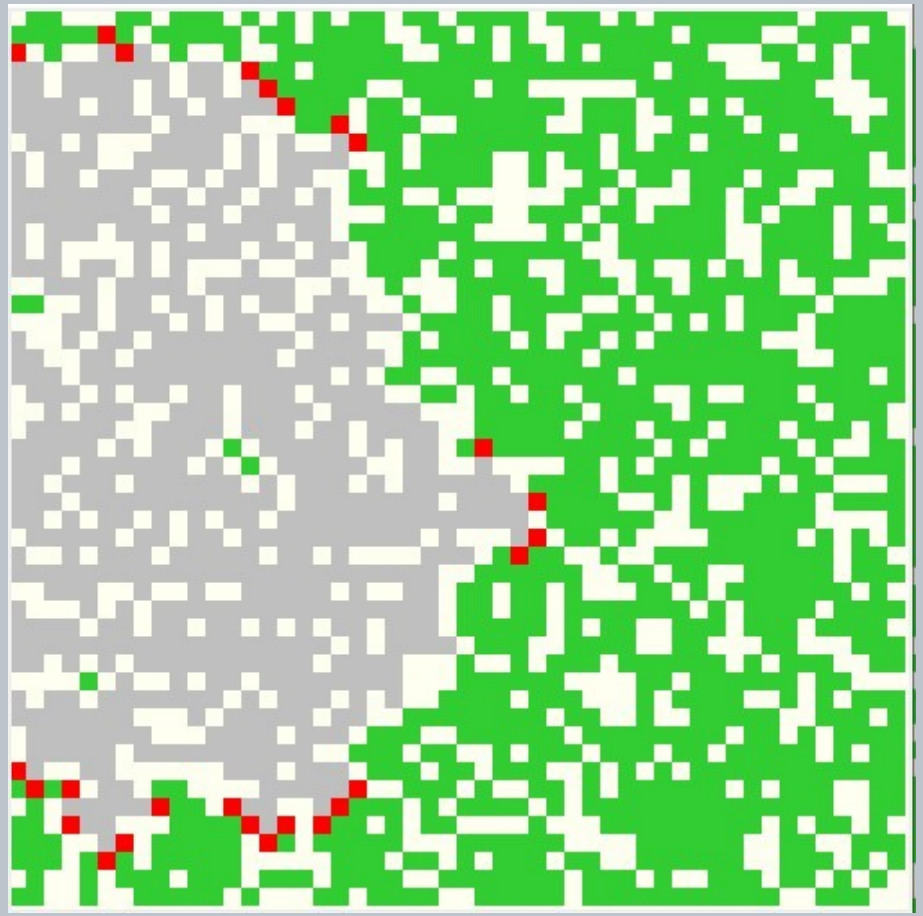
# Progression d’un feu de forêt



Forêt amazonienne brûlant

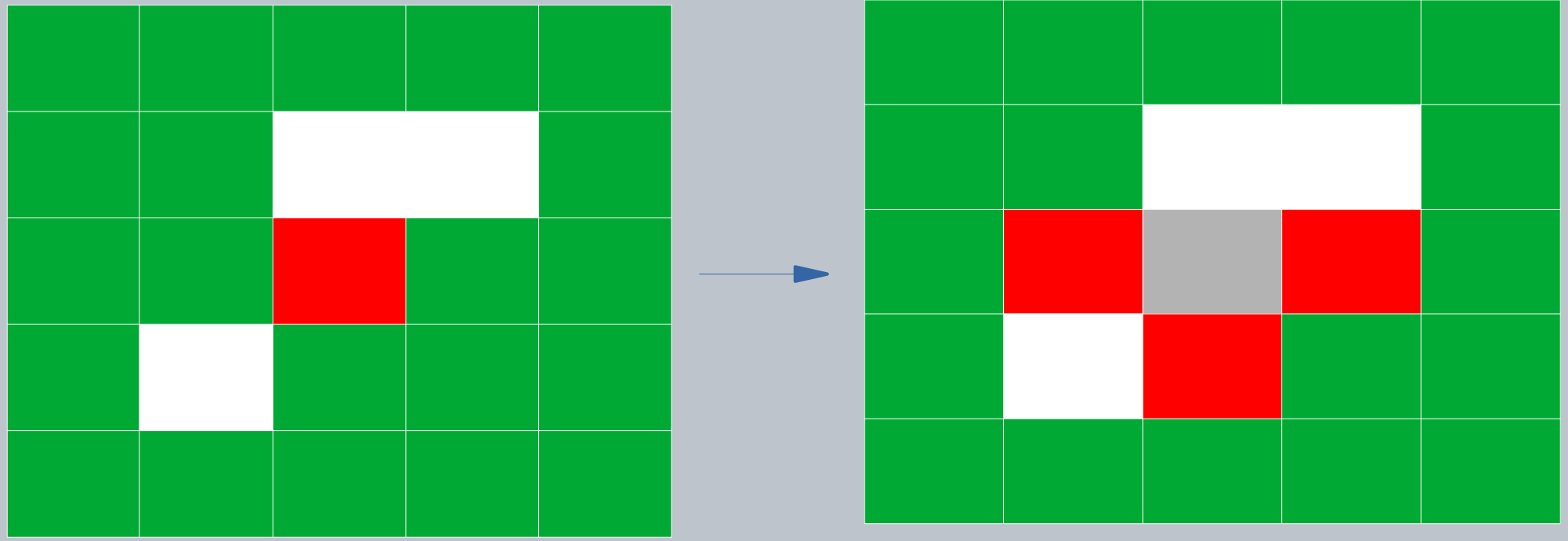
# Progression d’un feu de forêt avec un automate cellulaire





# Compréhension du programme n°1

## Comment fonctionne une itération ?



# Compréhension du programme n°1

```
def random_forest(p, n):
    units=[(line,col) for col in range(n) for line in range(n)]
    ntrees=int(n**2*p)
    trees=sample(units,ntrees)
    states=[[0]*n for _ in range(n)]
    for (i,j) in trees:
        states[i][j]=1
    return states
```

ntrees = nombre d’arbres

sample (units, ntrees) renvoie une liste de ntrees cellules de units

# Compréhension du programme n°1

```
def fill_cell(states, line, col):
        A=(unit*col, unit*line)
        B=(unit*(col+1), unit*(line+1))
        state=states[line][col]
        color=COLORS[state]
        cnv.create_rectangle(A, B, fill=color, outline='')
```

COLORS = ["ivory", "lime green", "red", "gray75"]

cnv.create_rectangle(.,.,.,.) permet enfin de dessiner un carré de coté 1

# Problèmes



- Tous les arbres ne brûlent pas à la même vitesse !!!

- Le programme n°1 ne représente que partiellement la réalité.

# Problématique

Dans quelles mesures est-il possible d’améliorer le programme d’un automate cellulaire représentant un feu de forêt afin de le rendre plus réaliste ?

# Dans quel but ?

L’objectif est de réussir à rendre ce programme utilisable pour les pompiers et même les populations avoisinant un feu de forêt.
Il serait intéressant de pouvoir prédire les premières zones touchées pour envisager une contre-attaque, mais aussi de planifier de possibles évacuations de la population.

# Dans quel but ?

Contre-feu  Évacuation  Mobilisation des pompiers



# Plan des modifications

- Modification n°1 : On tente de réutiliser le programme 1 avec plusieurs couleurs, afin de complexifier la végétation et l’évolution du feu.

- Modification n°2 : On essaye de prédire l’évolution du feu, en générant une liste contenant le temps auquel chacun des sommets brûlera. Ensuite on représente dans un second temps, le feu pour avoir un interprétation visuelle du résultat.

# Modification du programme - modification n°1

0 = rocher ne peut pas brûler

1 = forêt sèche = vert clair (= brûle en 1 itération )

2 = forêt = vert (= brûle en 2 itérations)

3 = forêt humide = vert foncé (= brûle en 3 itérations)

4 = eau = bleu (= brûle en 4/5 itérations)

5 =  feu = rouge

6 = cendre = gris

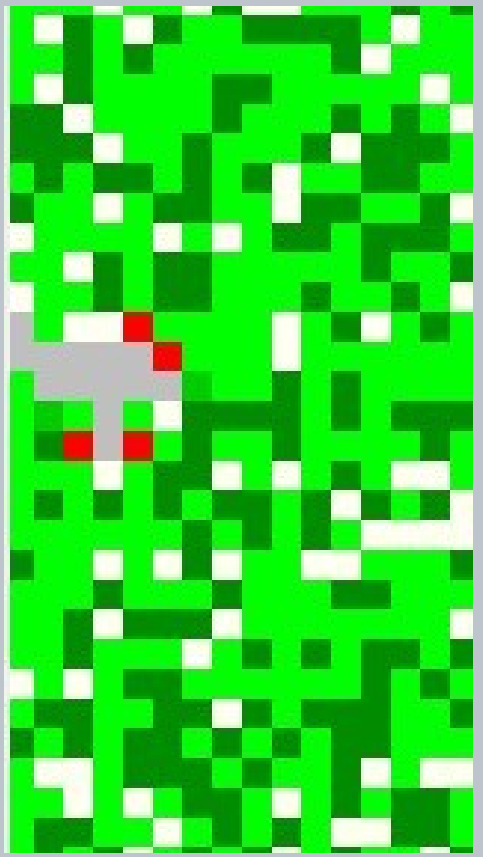
# Modification du programme - modification n°1

```
def random_forest(he,a,hu,ea, n):
    units=[(line,col) for col in range(n) for line in range(n)]
    na=int(n**2*a)
    nhe=int(n**2*he)
    nhu=int(n**2*hu)
    nea=int(n**2*ea)
    herbes=sample(units,nhe)
    arbres=sample(units,na)
    humide=sample(units,nhu)
    eau=sample(units,nea)
    states=[[0]*n for _ in range(n)]
    for (i,j) in herbes:
        states[i][j]=1
    for (i,j) in arbres:
        states[i][j]=2
    for (i,j) in humide:
        states[i][j]=3
    for (i,j) in eau:
        states[i][j]=4
    return states
```

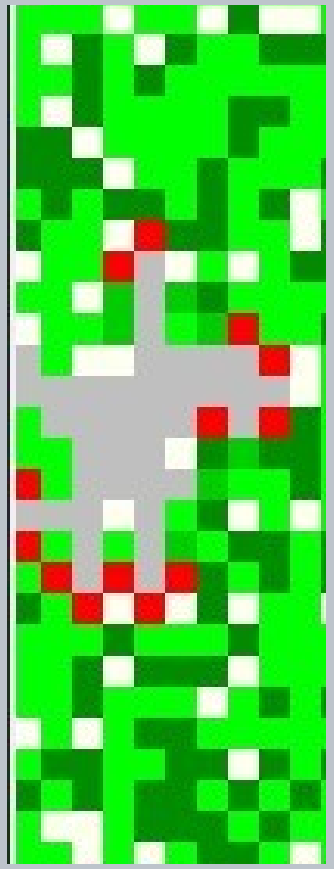


# Modification du programme - modification n°1

Avant :



Après :



# Modification du programme - modification n°1



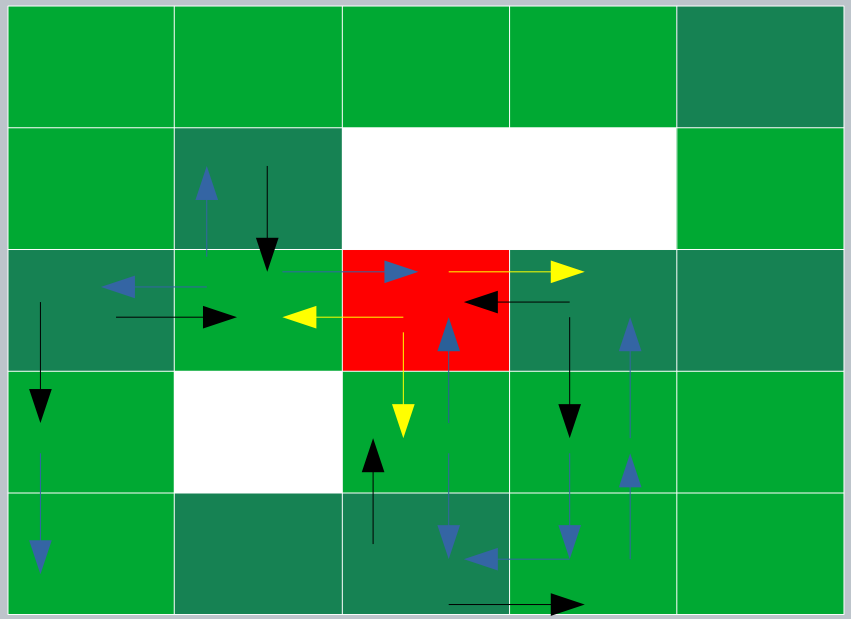
# Modification du programme - modification n°2

- On choisit, afin de faciliter le déroulement du programme, de **calculer préalablement le temps** qu'il faut pour qu'une cellule commence à brûler.

- On considère states comme un **graphe orienté et pondéré** de n*n sommets.

# Modification du programme - modification n°2

## Représentation du graphe



1 itération

2 itérations

direction du feu

# Modification du programme - modification n°2

```
def graph_pond_oriente(states):
    graph=dict()
    for i in range(n):
        for j in range(n):
            graph[(i,j)]={(a,b):states[a][b] for (a,b) in voisins(n, i, j)}
    return graph
```

```
def graph_pond_oriente2(states):
    graph=dict()
    for i in range(n):
        for j in range(n):
            graph[1*i+0.01*j]=[(1*a+0.01*b,states[a][b]) for
    return graph
```

f(x,y)=x+0.01*y est injective sur [0,99]*[0,99]

# Modification du programme - modification n°2

```
In [279]: graph=graph_pond_orien

In [280]: graph
Out[280]:
{(0, 0): {(0, 1): 0, (1, 0): 0},
 (0, 1): {(0, 0): 3, (0, 2): 0,
 (0, 2): {(0, 1): 0, (0, 3): 2,
 (0, 3): {(0, 2): 0, (0, 4): 2,
 (0, 4): {(0, 3): 2, (0, 5): 4,
 (0, 5): {(0, 4): 2, (0, 6): 0,
 (0, 6): {(0, 5): 4, (0, 7): 2,
 (0, 7): {(0, 6): 0, (0, 8): 4,
 (0, 8): {(0, 7): 2, (0, 9): 1,
 (0, 9): {(0, 8): 4, (1, 9): 4},
 (1, 0): {(0, 0): 3, (1, 1): 4,
```

```
In [277]: graph=graph_pond_oriente2(states)

In [278]: graph
Out[278]:
{0.0: [(0.01, 0), (1.0, 0)],
 0.01: [(0.0, 3), (0.02, 0), (1.01, 4)],
 0.02: [(0.01, 0), (0.03, 2), (1.02, 2)],
 0.03: [(0.02, 0), (0.04, 2), (1.03, 0)],
 0.04: [(0.03, 2), (0.05, 4), (1.04, 2)],
 0.05: [(0.04, 2), (0.06, 0), (1.05, 3)],
 0.06: [(0.05, 4), (0.07, 2), (1.06, 1)],
 0.07: [(0.06, 0), (0.08, 4), (1.07, 2)],
 0.08: [(0.07, 2), (0.09, 1), (1.08, 0)],
 0.09: [(0.08, 4), (1.09, 4)],
 1.0: [(0.0, 3), (1.01, 4), (2.0, 3)],
```

# Modification du programme - modification n°2

```
def dijkstra(graph, depart):
    infini=[]
    a_traiter = {k: float('inf') for k in graph}
    a_traiter[depart] = 0
    resultat = {}

    while a_traiter != {}:
        # Trouver le noeud avec la plus petite valeur
        k = min(a_traiter, key=a_traiter.get)#renvoie la plus petite valeur d

        # Mettre à jour les distances de ses voisins
        for v, d in graph[k]:
            if v not in resultat:
                if v in a_traiter:
                    a_traiter[v] = min(a_traiter[v], a_traiter[k] + d)

        resultat[k] = a_traiter[k]
        del a_traiter[k]
    for e in resultat:
        if resultat[e]>2000:
            infini.append(e)
    for e in infini:
        del resultat[e]

    return resultat
```

# Modification du programme - modification n°2

```
In [7]: resultat
Out[7]:
{0.0: 0,
 1.0: 2,
 2.0: 3,
 2.01: 6,
 3.0: 7,
 3.01: 8,
 2.02: 9,
 4.0: 9,
 3.02: 10,
 3.03: 11,
 4.01: 11,
 5.0: 11,
 5.01: 12,
 2.03: 13,
 3.04: 13,
 4.02: 13,
 4.03: 15,
 4.04: 15,
 5.02: 15,
 6.02: 16,
```

```
In [6]: runfile('C:/Users/Martin/Downloads/TIPE test.py', wdir='C:/
Users/Martin/Downloads')
[[2, 0, 4, 0, 3, 3, 2, 3, 3, 4], [2, 0, 0, 4, 2, 4, 0, 2, 4, 3], [1, 3,
3, 4, 0, 0, 3, 4, 3, 3], [4, 2, 2, 1, 2, 0, 2, 2, 3, 0], [2, 3, 3, 4, 2,
4, 2, 4, 4, 3], [2, 1, 3, 2, 2, 4, 3, 4, 2, 4], [0, 0, 1, 3, 0, 1, 2, 2,
0, 3], [0, 2, 3, 2, 2, 2, 3, 2, 1, 4], [3, 4, 2, 4, 3, 4, 4, 3, 4, 4],
[4, 3, 4, 3, 3, 2, 2, 4, 1, 3]]
```

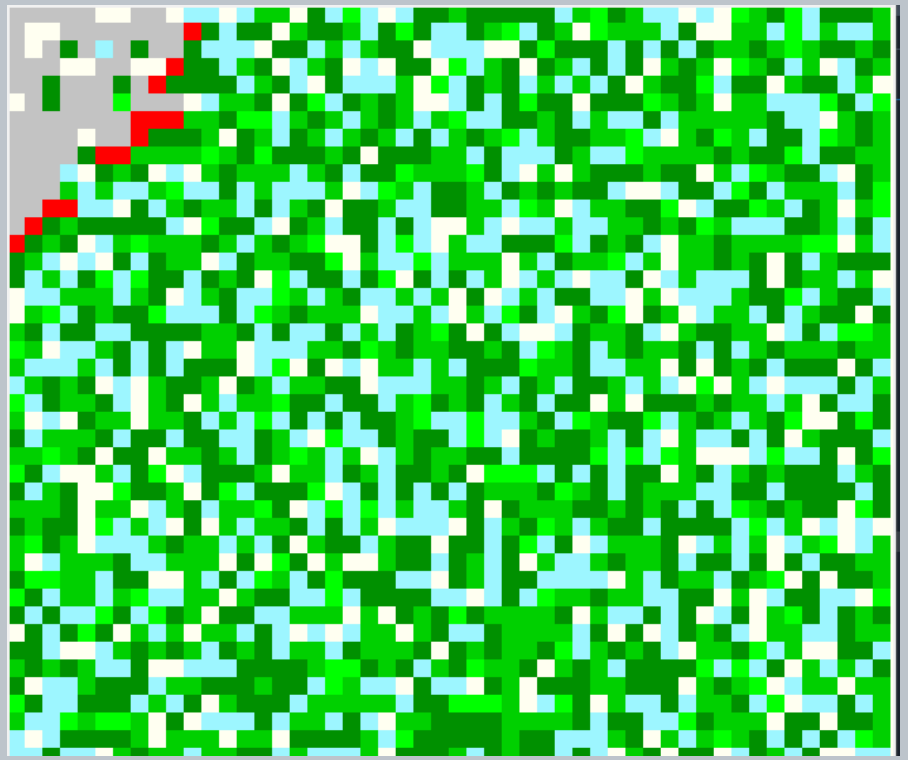
# Modification du programme - modification n°2

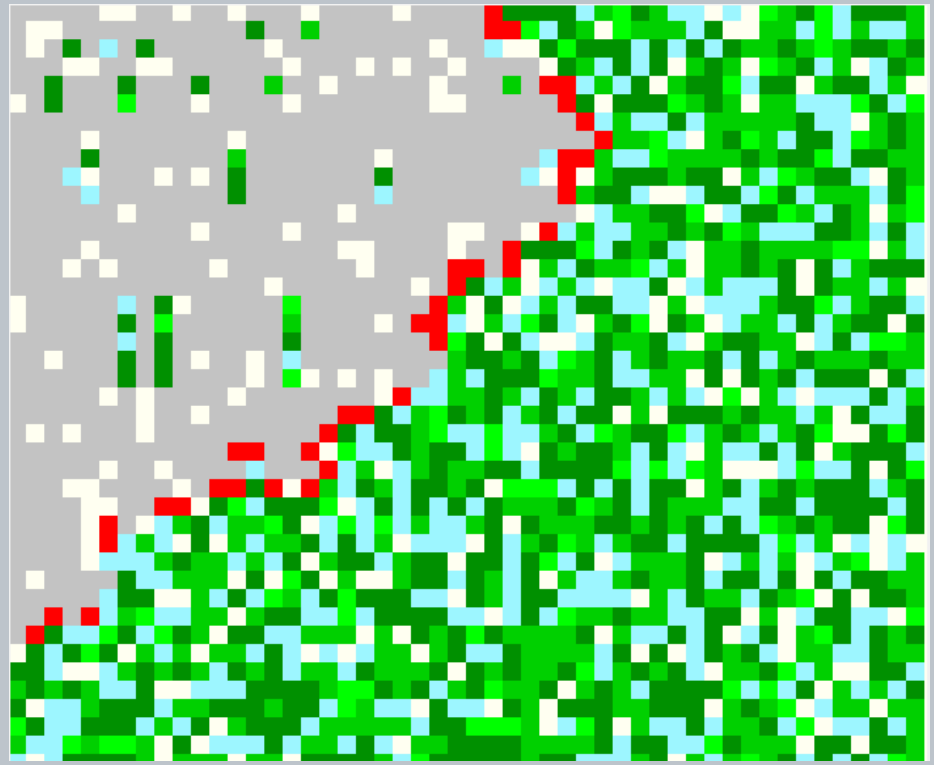
```
In [9]: L
Out[9]:
[[0.0],
 [],
 [1.0],
 [2.0],
 [],
 [],
 [2.01],
 [3.0],
 [3.01],
 [2.02, 4.0],
 [3.02],
 [3.03, 4.01, 5.0],
 [5.01],
 [2.03, 3.04, 4.02],
 [],
 [4.03, 4.04, 5.02],
 [6.02],
 [1.03, 5.03, 5.04],
 [],
 [1.04, 4.05, 6.03, 7.02],
 [],
```

```
In [10]: L2
Out[10]:
[[0.0],
 [],
 [],
 [],
 [1.0, 2.0],
 [],
 [],
 [2.01],
 [],
 [],
 [3.01],
 [3.0, 4.0],
 [2.02, 3.02, 3.03],
 [5.0, 5.01],
 [4.01],
 [3.04],
 [2.03, 4.02, 5.02],
 [4.04, 6.02],
 [],
 [4.03, 5.03, 5.04],
 [],
 [1.03, 1.04],
 [6.03, 7.02],
```

# Modification du programme - modification n°2

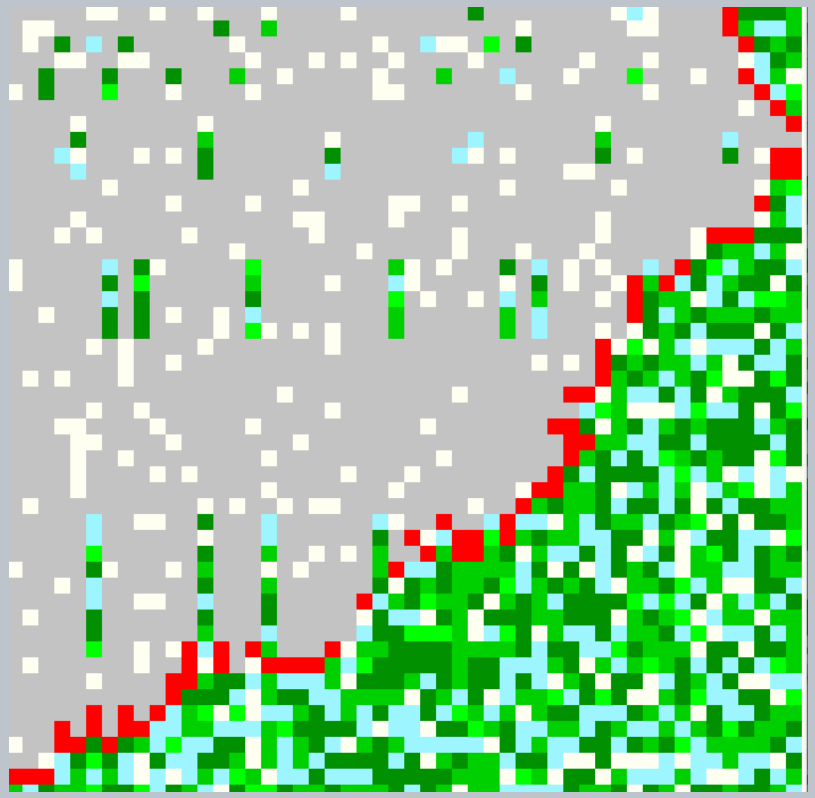
Plus proche de la réalité ?

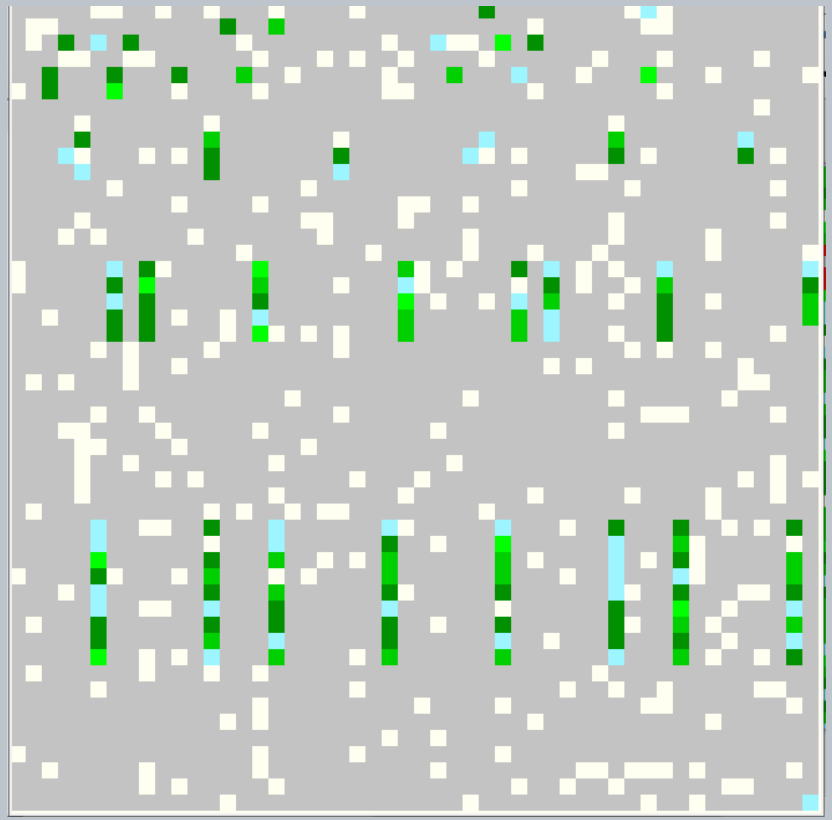




# Modification du programme - modification n°2

Plus proche de la réalité ?





# Modification du programme - modification n°2

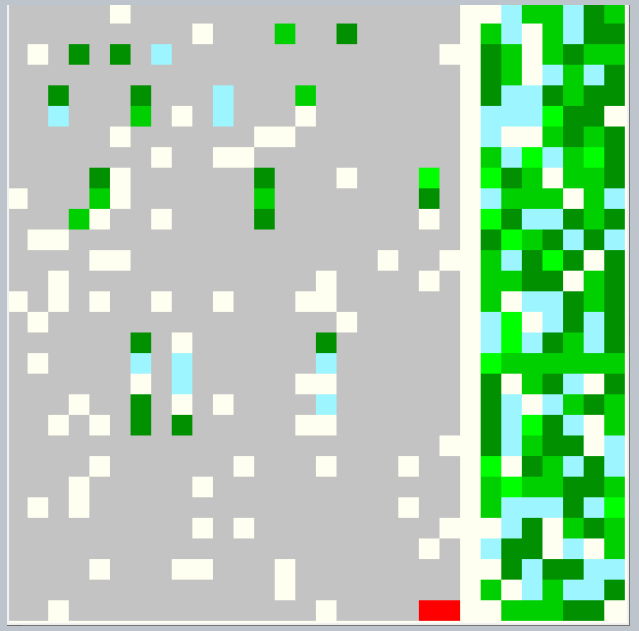
- Il existe cependant des incertitudes !!!
- Néanmoins, on peut les interpréter comme le hasard tel qu'il existe dans la nature, car la propagation d'un feu de forêt est un système **chaotique**.

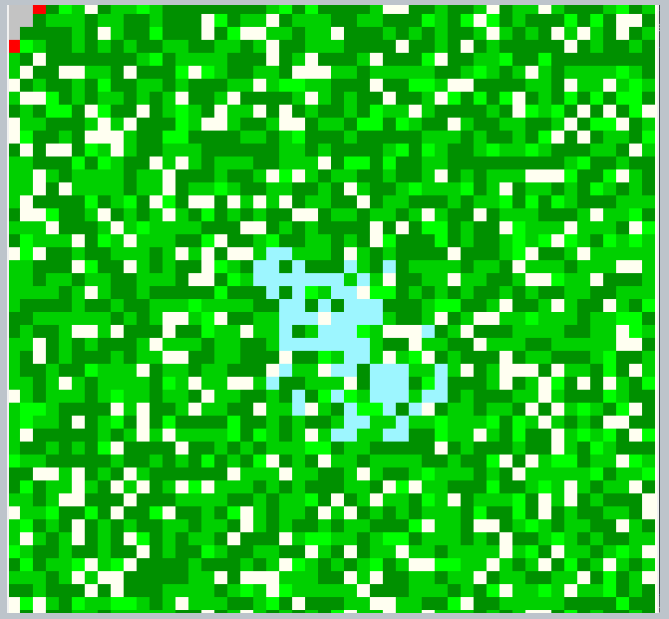
```
In [33]: int(100*7.03-100*int(7.03))
Out[33]: 3
```

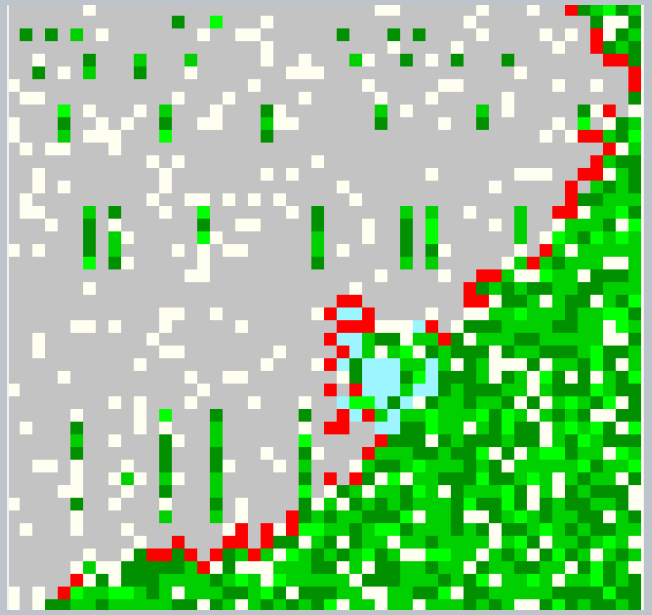
```
In [46]: int(100*4.02-100*int(4.02))
Out[46]: 1

In [47]: 100*4.02
Out[47]: 401.99999999999994
```

# Utilisation du programme







# Complexité

- La complexité est en **$0(n^3)$** pour le programme n°1
- Notre programme est de complexité **$0(ln(n)*n^3)$**

- En effet, il est en $0(m*n^2)$ pour le programme n°2 où m est la valeur de la dernière itération à laquelle une cellule va brûler, cela correspond à la valeur maximale du dictionnaire « resultat ». De plus, expérimentalement, on trouve que **m est de l'ordre de n*ln(n)**.

# Conclusion

- Nous avons réussi à rendre plus **réaliste** et plus **utilisable** la propagation de ce feu de forêt.

- D’une part en **complexifiant l’évolution et la transition entre les états des cellules**.

- D’une autre en permettant de **prévoir** l’évolution du feu

# Conclusion

- L’objectif est atteint car notre dictionnaire « résultat » renvoie à lui seul à chacune des cellules, associées au moment auquel elles vont commencer à brûler.
- Néanmoins, on ne peut rivaliser avec des programmes à plus gros budget et aux IA telles que celles de la référence [6] de la bibliographie commenté.

# Annexe
# Programme modifié

```python
from random import sample
from tkinter import Tk, Canvas

COLORS=["ivory","green1", "green3","green4", "cadetblue1","red", "gray75"]

def random_forest(he,a,hu,ea, n):
    units=[(line,col) for col in range(n) for line in range(n)]
    na=int(n**2*a)
    nhe=int(n**2*he)
    nhu=int(n**2*hu)
    nea=int(n**2*ea)
    herbes=sample(units,nhe)
    arbres=sample(units,na)
    humide=sample(units,nhu)
    eau=sample(units,nea)
    states=[[0]*n for _ in range(n)]
    for (i,j) in herbes:
        states[i][j]=1
    for (i,j) in arbres:
        states[i][j]=2
    for (i,j) in humide:
        states[i][j]=3
    for (i,j) in eau:
        states[i][j]=4
    return states

def modifterrain(states,j):
    for i in range(n):
        states[i][j]=0
    return states
```

```python
def voisins(n, i, j):
    return [(a,b) for (a, b) in
            [ (i, j-1),(i-1, j), (i, j+1),(i+1,j)]
            if a in range(n) and b in range(n) and states[a][b]!=0]


def graph_pond_oriente(states):
    graph=dict()
    for i in range(n):
        for j in range(n):
            graph[(i,j)]={(a,b):states[a][b] for (a,b) in voisins(n, i, j)}
    return graph

def graph_pond_oriente2(states):
    graph=dict()
    for i in range(n):
        for j in range(n):
            graph[1*i+0.01*j]=[(1*a+0.01*b,states[a][b]) for (a,b) in voisins(n
    return graph
```

# Annexe
# Programme modifié

```python
def dijkstra(graph, depart):
    infini=[]
    a_traiter = {k: float('inf') for k in graph}
    a_traiter[depart] = 0
    resultat = {}

    while a_traiter != {}:
        k = min(a_traiter, key=a_traiter.get)
        for v, d in graph[k]:
            if v not in resultat:
                if v in a_traiter:
                    a_traiter[v] = min(a_traiter[v], a_traiter[k] + d)

        resultat[k] = a_traiter[k]
        del a_traiter[k]
    for e in resultat:
        if resultat[e]>2000:
            infini.append(e)
    for e in infini:
        del resultat[e]

    return resultat
```

```python
def liste(resultat):
    m=max(resultat.values())
    n=int(m+1)
    L=[[] for i in range(n)]
    valeur=list(resultat.values())
    sommet=list(resultat.keys())
    for i in range(n):
        for j in range(len(valeur)):
            if valeur[j]==i:
                L[i].append(sommet[j])
    return L

def findecombustion(resultat,states):
    resultat2={}
    for e in resultat:
        if e!=0.00:
            resultat2[e]=resultat[e]+states[int(e)][int(100*e-100*int(e))]
        else:
            resultat2[e]=resultat[e]
    return resultat2
```

# Annexe
# Programme modifié

```
def update_states2(states,it):
    for e in L[it]:
        states[int(e)][int(100*e-100*int(e))]=5
    for e in L2[it]:
        states[int(e)][int(100*e-100*int(e))]=6

def fill_cell(states, line, col):
        A=(unit*col, unit*line)
        B=(unit*(col+1), unit*(line+1))
        state=states[line][col]
        color=COLORS[state]
        cnv.create_rectangle(A, B, fill=color, outline='')

def fill(states):
    n=len(states)
    for line in range(n):
        for col in range(n):
            fill_cell(states, line, col)


def propagate(it):
    m=max(resultat.values())
    n=int(m+1)
    if it<m:
        update_states2(states,it)
        cnv.delete("all")
        fill(states)
        cnv.after(200,propagate,it+1)
```

```
he= 0.40
hu= 0.40
a=0.60
ea=0.25
n=50
unit=13
k=22
root = Tk()
cnv = Canvas(root, width=unit*n, height=unit*n, background="ivory")
cnv.pack()

states=random_forest(he,a,hu,ea, n)
#states=modifterrain(states,k)

graph=graph_pond_oriente2(states)
depart=0.00
resultat=dijkstra(graph,depart)
resultat2=findecombustion(resultat,states)
L=liste(resultat)
L2=liste(resultat2)

states[0][0]=5

fill(states)
propagate(0)

root.mainloop()
```